MLOps Portfolio: Interview Questions

Updated 29 October 2026. Read the tools primer and architecture guide first. These answers describe the repaired code and measured evidence; adapt the wording to your own understanding.

1. Start with the project

Interviewer: What did you build?

You: I built a Python MLOps demonstration around one TF-IDF and logistic-regression spam classifier. It has a FastAPI serving image, a Kafka inference flow and Kubernetes manifests. A separate Terraform exercise creates S3 and DynamoDB resources in LocalStack. There is also an optional Bedrock interpretation command, whose live AWS path remains unverified.

Interviewer: Is this a production spam detector?

You: The engineering paths are testable, but the model uses only 40 curated messages. I describe it as an operations demonstration, with a small fixture for predictable runs. I would need a representative corpus, independent evaluation, monitoring, access controls and operational targets before proposing production use.

Interviewer: What did your audit actually change?

You: It added model-aware readiness and request limits, made Kafka wait for confirmed output before acknowledging input, repaired Python 3.12 client compatibility, and added training-fold-only evaluation. Automated checks exercise failures as well as normal predictions. The verification record separates local Python results, real infrastructure runs and mocked Bedrock checks.

2. Explain the machine learning without hiding behind terms

Interviewer: How does TF-IDF help?

You: It turns words into numeric features. A word gets more weight when it appears in this message but is uncommon across the training messages. The vectorizer learns its vocabulary and frequency statistics during fitting. At prediction time it applies that stored mapping, rather than learning from the new message.

> Analogy: if every shop receipt says "item," that word tells me little. A rarer product name helps distinguish one receipt from the rest. TF-IDF gives more weight to those distinguishing terms.

Interviewer: Why logistic regression?

You: It is a fast linear baseline for sparse text features. It learns a weight for each feature and converts the weighted score into class probabilities. That is sufficient for demonstrating serving and streaming. A more complex classifier would not repair missing acknowledgements or false readiness.

Interviewer: Why save a Pipeline?

You: The classifier expects exactly the feature mapping used in training. The Pipeline keeps TF-IDF and logistic regression together, so inference uses the fitted vocabulary and weights. Saving only the final estimator would make it easy to supply incompatible vectors.

Interviewer: You achieved 100% accuracy?

You: That was training accuracy on all 40 fixture records. Five-fold evaluation averaged 77.5% accuracy, with 12.25 percentage points of fold dispersion. Mean spam recall was 65%. I report both so that memorising this tiny training set is not mistaken for generalisation.

Interviewer: Is 77.5% a deployable result?

You: It is a diagnostic on a small curated fixture. Each fold tests only eight messages. Its standard deviation is not a confidence interval, and none of these figures estimates inbox performance reliably. One separate eight-message audit split scored 100%, which illustrates why I would not choose a flattering split as the headline.

Interviewer: Where would leakage occur?

You: If I fitted the vectorizer on the full corpus before cross-validation, held-out messages would influence vocabulary and IDF. Here, cross-validation fits the entire Pipeline separately on each training fold. After evaluation, a fresh Pipeline fits all 40 records to create the serving artifact.

Interviewer: What does the confidence field mean?

You: It is the maximum probability returned by logistic regression. The project has no calibration evaluation. A score of 0.66 is not a validated 66% correctness rate on future messages. A message with no known tokens receives a warning because its prediction is driven by the model's intercept rather than recognised word evidence.

3. Explain the API and the repaired readiness behavior

Interviewer: Why are health and readiness separate?

You: `/health` says the process can answer. `/ready` says it has a usable model. The audit removed the assumption that a green process heartbeat proves inference is available. Missing or corrupt artifacts now make readiness and prediction return 503, while liveness can remain 200.

> Analogy: a restaurant can answer its phone while its oven is broken. Liveness checks the phone; readiness checks whether the kitchen can take orders.

Interviewer: Why change lazy loading?

You: Loading during the first request created a gap between healthy startup and working inference. The lifespan handler now loads and verifies the artifact once before normal requests. Each process still has its own model; this is not shared memory across Kubernetes replicas.

Interviewer: What does validation reject?

You: The API requires one strict string named `text`, with one to 10,000 characters and at least one non-whitespace character. Missing values, nulls, numbers, blank text, excess length and extra fields return 422. It preserves the original message. Unknown-vocabulary text is allowed but clearly marked with `no_known_tokens`.

Interviewer: Are you protected against every harmful request?

You: These checks improve the input contract. They do not supply authentication, rate limits, a global HTTP body-size policy or a production security review. The project does not claim those features.

Interviewer: Can you safely load any uploaded pickle?

You: No. Joblib uses pickle-based serialization, which can execute code. The service loads the trusted artifact produced by its build. A production model supply chain would need provenance, controlled publication and compatible runtime versions.

4. Trace the streaming guarantee

Interviewer: What happens to one message?

You: The producer writes to `messages`. The consumer calls `/predict`, publishes the result to `predictions`, waits for its send future to confirm delivery, then commits that input record's next offset. It disables automatic commits and supplies an explicit partition/offset map.

Interviewer: Why commit offset 18 after processing offset 17?

You: A committed offset names the next record to consume. For partition 2 at offset 17, the consumer confirms the output and commits partition 2 at 18. It does not accidentally acknowledge a later fetched record or another partition.

Interviewer: What was wrong with flush alone?

You: The old flow did not inspect the individual send future, while input offsets were committed automatically. In fault injection, a failed publication was not surfaced by that control flow. Waiting on `send(...).get(...)` observes broker failure, and the commit happens only after success.

Interviewer: Is it exactly once now?

You: No. It is at least once. A crash after output acknowledgement but before input commit can produce the same output again on restart. `messages:2:17` remains the stable event ID on replay. Downstream deduplication could use it, but that deduplicator is not implemented.

Interviewer: What if a message is malformed?

You: The consumer fails without committing it. That makes the failure visible and preserves the record for retry, but can block the partition until an operator intervenes. A dead-letter topic with a documented handling policy would be a future improvement. Silently skipping the message would hide data loss.

Interviewer: What about retries?

You: Connection failures, timeouts, 429 and server errors have bounded retries. A permanent 400 or 422 is not retried as if waiting longer could fix the payload. Persistent failure leaves the input unacknowledged. Retry safety and duplicate handling are separate concerns.

5. Infrastructure questions

Interviewer: What does Docker make reproducible?

You: It packages the application, supported dependencies and trained artifact. The same serving image can run behind Kafka or on Kubernetes. I do not claim eternal binary reproducibility: the base tag and some transitive dependencies can still change. Reproducible behavior needs more than a Dockerfile's presence.

Interviewer: Why does Kafka advertise kafka:9092?

You: That is its name inside the Compose network. Clients learn advertised listener addresses from the broker. Returning localhost to another container would send that client back to itself. A separate host listener would be needed for general host-client use; publishing port 9092 alone does not fix advertised metadata.

Interviewer: What do the Kubernetes probes do?

You: Startup allows time for model loading. Readiness controls whether the pod should receive traffic and checks `/ready`. Liveness checks `/health` to decide whether the process needs restarting. The Deployment requests two replicas; the integration check verifies a deleted pod is replaced.

Interviewer: What is KRaft?

You: Kafka's controllers manage metadata using Raft consensus, replacing the separate ZooKeeper dependency. Our single node runs both broker and controller roles. That simplifies a local demonstration but does not prove controller-quorum or multi-broker fault tolerance.

Interviewer: What is the limit of the Kubernetes test?

You: It verifies local rollout, real HTTP predictions and replacement of a deleted pod. It does not measure cross-node failover, production throughput or a latency service-level objective. Two replicas on one local cluster are not a disaster-recovery result.

Interviewer: Is Terraform provisioning the Kubernetes cluster?

You: No. The supplied Terraform creates only an S3 bucket and DynamoDB table in LocalStack. The local Kubernetes cluster is created separately for the deployment test. The API does not write artifacts or predictions to those cloud resources.

Interviewer: What did your Bedrock test prove?

You: It proved request construction, response parsing and invalid-input rejection with a mocked SDK response. The CLI obtains a real classifier result before making its optional LLM call. There is no live Bedrock result to claim. Even a successful call would supply an interpretation, not measured feature attribution from logistic-regression coefficients.

6. Answer the advanced follow-ups honestly

Interviewer: How would you harden this next?

You: I would first choose representative data and an independent evaluation protocol. For operations, I would define throughput and latency targets, measure consumer lag, add a poison-record policy, design deduplication, authenticate requests and control artifact publication. Each change should have an acceptance check. They are plans, not completed features.

Interviewer: How does this compare with a full MLOps platform?

You: It demonstrates a tested subset: packaging, serving, stream integration, local orchestration and emulated resource provisioning. It has no production monitoring, drift detector, model registry, retraining scheduler, canary rollout or rollback controller. I would present its scope clearly and show the passing execution evidence.

Practice

Explain one Kafka record from offset 17 through output acknowledgement and offset 18 commit. Sketch which modules are connected and which are separate. Explain why 100% training accuracy and green liveness are insufficient. Then reproduce the CLI and the failure tests without memorising the wording above.

Sources: Actual implementation | How it works | Tools primer | Verification evidence